Part 4: Advanced LangChain

LCEL, Runnables, Advanced Patterns & Production Techniques

Prerequisites: Complete Parts 1-3
Concepts Covered: 50+
Reading Time: 5-6 hours
Focus: Production-ready patterns and advanced techniques

Table of Contents

17. LCEL (LangChain Expression Language)
18. Runnables - Composable Components
19. Output Parsers - Structured Responses
20. Callbacks - Event Tracking
21. Caching - Performance Optimization
22. Advanced Retrievers
23. Advanced Chain Patterns

Chapter 17: LCEL (LangChain Expression Language)

17.1 What Is LCEL?

LCEL = Modern, declarative way to build chains (introduced LangChain v0.1)

Think of LCEL as:

- 🔗 **Pipe Operator**: Chain components with `.pipe()`
- 🧩 **Composable**: Mix and match components easily
- ⚡ **Optimized**: Built-in streaming, async, batching
- 📝 **Readable**: Code reads like natural flow

Old Way (LangChain v0.0)

```
import { LLMChain } from "langchain/chains";

const chain = new LLMChain({ llm, prompt });
const result = await chain.call({ input: "Hello" });
```

New Way (LCEL)

```
const chain = prompt | llm | outputParser;
const result = await chain.invoke({ input: "Hello" });
```

Benefits of LCEL:

- ☑ More concise (less boilerplate)
- ☑ Better TypeScript support
- ☑ Streaming built-in
- ☑ Easier to compose and reuse
- ☑ Automatic retry and fallbacks

17.2 Basic LCEL Chain

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";
import { StringOutputParser } from "@langchain/core/output_parsers";

// 1. Create components
const prompt = ChatPromptTemplate.fromTemplate(
  "Tell me a joke about {topic}"
);

const model = new ChatOpenAI({ modelName: "gpt-4o-mini" });

const parser = new StringOutputParser();

// 2. Compose with pipe
const chain = prompt.pipe(model).pipe(parser);
// Or shorthand:
// const chain = prompt | model | parser;

// 3. Invoke
const result = await chain.invoke({ topic: "expenses" });

console.log(result);
// "Why did the expense report go to therapy? Because it had too many issues that
needed to be reconciled!"
```

Flow:

```
{ topic: "expenses" }  →  Prompt  →  LLM  →  Parser  →  "Why did..."
```

17.3 LCEL Operators

pipe() (Sequential Composition)

```
const chain = componentA.pipe(componentB).pipe(componentC);
```

```
// Equivalent to:
// componentA → componentB → componentC
```

| (Shorthand Pipe)

```
const chain = componentA | componentB | componentC;

// Same as pipe(), more concise
```

.invoke() (Single Input)

```
const result = await chain.invoke({ input: "Hello" });
// Processes one input, returns one output
```

.batch() (Multiple Inputs)

```
const results = await chain.batch([
  { input: "Hello" },
  { input: "Hi" },
  { input: "Hey" }
]);
// Processes multiple inputs in parallel
// Returns: ["Response 1", "Response 2", "Response 3"]
```

.stream() (Streaming Output)

```
const stream = await chain.stream({ input: "Tell a story" });

for await (const chunk of stream) {
  process.stdout.write(chunk);  // "Once", " upon", " a", " time", ...
}
```

17.4 RunnablePassthrough (Identity)

Pass input through unchanged (useful for parallel flows)

```
import { RunnablePassthrough } from "@langchain/core/runnables";

const chain = RunnablePassthrough.assign({
  uppercase: (input) => input.text.toUpperCase()
});
```

```
const result = await chain.invoke({ text: "hello" });

console.log(result);
// { text: "hello", uppercase: "HELLO" }
// ↑ Original text + new uppercase field
```

Use case: Add computed fields without losing original input

17.5 RunnableLambda (Custom Function)

Wrap custom function as Runnable

```
import { RunnableLambda } from "@langchain/core/runnables";

// Custom function
const addPrefix = (text) => `[USER] ${text}`;

// Wrap as Runnable
const prefixer = RunnableLambda.from(addPrefix);

// Use in chain
const chain = prefixer.pipe(model);

const result = await chain.invoke("Hello");
// LLM receives: "[USER] Hello"
```

17.6 RunnableBranch (Conditional)

Choose different paths based on condition

```
import { RunnableBranch } from "@langchain/core/runnables";

const branch = RunnableBranch.from([
  // [condition, runnable]
  [(input) => input.type === "question", questionChain],
  [(input) => input.type === "command", commandChain],
  defaultChain  // Fallback
]);

// Route to appropriate chain
await branch.invoke({ type: "question", text: "..." });  // → questionChain
await branch.invoke({ type: "command", text: "..." });  // → commandChain
await branch.invoke({ type: "other", text: "..." });  // → defaultChain
```

Use case: Routing based on input type (like intent classification)

17.7 RunnableParallel (Parallel Execution)

Run multiple chains in parallel

```
import { RunnableParallel } from "@langchain/core/runnables";

const parallel = RunnableParallel.from({
  summary: summaryChain,
  sentiment: sentimentChain,
  keywords: keywordChain
});

const results = await parallel.invoke({ text: "..." });

console.log(results);
// {
//   summary: "Summary of text...",
//   sentiment: "Positive",
//   keywords: ["expense", "report", "approval"]
// }
// ↑ All 3 chains run simultaneously
```

Benefits:

- ☑ Faster (parallel vs sequential)
- ☑ Multiple perspectives on same input

17.8 Real Example: ai-langx/ with LCEL

Potential refactor of expense categorization

```
// File: src/chains/categorizationLCEL.js
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { ChatOpenAI } from "@langchain/openai";
import { StructuredOutputParser } from "@langchain/core/output_parsers";
import { RunnablePassthrough } from "@langchain/core/runnables";
import { z } from "zod";

// Output schema
const schema = z.object({
  category: z.enum(["Food", "Transport", "Shopping", "Entertainment", "Bills",
"Healthcare", "Education", "Other"]),
  confidence: z.number().min(0).max(1),
  reasoning: z.string()
});

// Parser
const parser = StructuredOutputParser.fromZodSchema(schema);

// Prompt
const prompt = ChatPromptTemplate.fromMessages([
  ["system", `Categorize expenses into: Food, Transport, Shopping, Entertainment,
```

```
Bills, Healthcare, Education, Other.

{format_instructions}`],
  ["human", "Description: {description}\nAmount: ₹{amount}"]
]);

// LLM
const model = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0
});

// Build chain with LCEL
export const categorizationChain = RunnablePassthrough.assign({
  format_instructions: () => parser.getFormatInstructions()
})
  .pipe(prompt)
  .pipe(model)
  .pipe(parser);

// Usage
const result = await categorizationChain.invoke({
  description: "Lunch at McDonald's",
  amount: 350
});

console.log(result);
// {
//   category: "Food",
//   confidence: 0.95,
//   reasoning: "Restaurant meal clearly falls under Food category"
// }
```

Equivalent old way (much longer):

```
// Old LangChain v0.0 syntax
const formatInstructions = parser.getFormatInstructions();
const promptInstance = await prompt.formatMessages({
  description: "Lunch",
  amount: 350,
  format_instructions: formatInstructions
});
const response = await model.invoke(promptInstance);
const parsed = await parser.parse(response.content);
```

17.9 LCEL Streaming

Stream tokens as they're generated

```
const chain = prompt | model | parser;

// Stream results
const stream = await chain.stream({ input: "Write a long story" });

for await (const chunk of stream) {
  process.stdout.write(chunk);  // Print each token as it arrives
}
```

In Express API:

```
router.post('/chat/stream', async (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');

  const chain = prompt | model | parser;
  const stream = await chain.stream({ input: req.body.message });

  for await (const chunk of stream) {
    res.write(`data: ${JSON.stringify({ chunk })}\n\n`);
  }

  res.write('data: [DONE]\n\n');
  res.end();
});
```

17.10 LCEL Best Practices

1. Use Pipe for Readability

```
// ☑ Good: Clear flow
const chain = prompt | model | parser;

// ✕ Bad: Nested
const chain = parser.pipe(model.pipe(prompt));
```

2. Type Safety

```
// ☑ Good: Type-safe with Zod
const parser = StructuredOutputParser.fromZodSchema(
  z.object({
    result: z.string()
  })
);
```

```
// Result is typed: { result: string }
```

3. Reusable Components

```
// Create reusable pieces
const summarizer = prompt | model | parser;
const translator = translationPrompt | model | parser;

// Compose into larger chains
const summarizeAndTranslate = summarizer | translator;
```

☑ **You now understand LCEL!**

Chapter 18: Runnables - Composable Components

18.1 What Are Runnables?

Runnable = Interface for composable components (everything in LCEL is a Runnable)

All these are Runnables:

- Prompts
- LLMs
- Output parsers
- Chains
- Tools
- Retrievers

Standard interface:

- `.invoke()` - Single input
- `.batch()` - Multiple inputs
- `.stream()` - Streaming output

18.2 Creating Custom Runnables

```
import { Runnable } from "@langchain/core/runnables";

class ExpenseValidator extends Runnable {
  async invoke(input) {
    const { amount, category } = input;

    // Validation logic
    if (amount <= 0) {
      throw new Error("Amount must be positive");
    }
```

```
    if (!["Food", "Transport", "Shopping"].includes(category)) {
      throw new Error("Invalid category");
    }

    // Return validated input
    return { amount, category, validated: true };
  }

  // Batch implementation
  async batch(inputs) {
    return Promise.all(inputs.map(input => this.invoke(input)));
  }
}

// Use in chain
const validator = new ExpenseValidator();
const chain = validator | saveToDatabase;

await chain.invoke({ amount: 500, category: "Food" });
```

18.3 RunnableSequence

Chain runnables sequentially (what `.pipe()` creates)

```
import { RunnableSequence } from "@langchain/core/runnables";

const chain = RunnableSequence.from([
  prompt,
  model,
  parser
]);

// Equivalent to:
// const chain = prompt | model | parser;
```

18.4 RunnableMap (Parallel with Keys)

Run multiple runnables in parallel, return keyed results

```
import { RunnableMap } from "@langchain/core/runnables";

const map = RunnableMap.from({
  categorize: categorizationChain,
  validate: validationChain,
  enrich: enrichmentChain
});

const results = await map.invoke({ description: "Lunch", amount: 500 });
```

```
console.log(results);
// {
//   categorize: { category: "Food", confidence: 0.9 },
//   validate: { valid: true },
//   enrich: { merchantType: "Restaurant", mealType: "Lunch" }
// }
```

18.5 Fallbacks (Error Handling)

Try primary, fall back to secondary on error

```
const primary = gpt4Chain;
const fallback = gpt35Chain;

const chainWithFallback = primary.withFallbacks({
  fallbacks: [fallback]
});

// If GPT-4 fails (rate limit, timeout), automatically tries GPT-3.5
const result = await chainWithFallback.invoke({ input: "..." });
```

Multiple fallbacks:

```
const chain = gpt4Chain.withFallbacks({
  fallbacks: [
    gpt4oMiniChain,  // Try this first
    gpt35Chain,  // Then this
    cachedResponseChain  // Finally this
  ]
});
```

18.6 Retry Logic

Automatically retry on failure

```
const chain = (prompt | model | parser).withRetry({
  stopAfterAttempt: 3,  // Max 3 attempts
  waitExponential: 2  // Wait 2^n seconds between retries
});

// Automatically retries on:
// - Network errors
// - Rate limits (429)
// - Timeouts
```

18.7 Configurable Runnables

Make runnables configurable at runtime

```
const chain = (prompt | model | parser).withConfig({
  runName: "ExpenseCategor ization",  // LangSmith trace name
  tags: ["categorization", "production"],
  metadata: { userId: "123", version: "v2" }
});

const result = await chain.invoke({ input: "..." });
// Traces logged with name, tags, metadata
```

☑ **You now understand Runnables!**

Chapter 19: Output Parsers - Structured Responses

19.1 Why Output Parsers?

Problem: LLM returns unstructured text

```
const response = await llm.invoke("Categorize: Lunch ₹500");
console.log(response.content);
// "The category is Food and the amount is ₹500."  ← String, not structured
```

Solution: Output parsers extract structured data

```
const parser = StructuredOutputParser.fromZodSchema(...);
const result = await parser.parse(response.content);
console.log(result);
// { category: "Food", amount: 500 }  ← Structured object!
```

19.2 StringOutputParser (Simple)

Extract plain text (strips message wrapper)

```
import { StringOutputParser } from "@langchain/core/output_parsers";

const parser = new StringOutputParser();

const chain = prompt | model | parser;

const result = await chain.invoke({ input: "Say hi" });
console.log(result);
// "Hello! How can I help you?"  ← Plain string
```

19.3 Structured OutputParser (Zod)

Parse into validated object

```
import { StructuredOutputParser } from "@langchain/core/output_parsers";
import { z } from "zod";

const parser = StructuredOutputParser.fromZodSchema(
  z.object({
    category: z.string(),
    amount: z.number(),
    isValid: z.boolean()
  })
);

// Get format instructions for LLM
const formatInstructions = parser.getFormatInstructions();

const prompt = ChatPromptTemplate.fromTemplate(`
Analyze this expense: {input}

{format_instructions}
`);

const chain = prompt | model | parser;

const result = await chain.invoke({
  input: "Lunch ₹500",
  format_instructions: formatInstructions
});

console.log(result);
// { category: "Food", amount: 500, isValid: true }
```

19.4 JsonOutputParser (Simple JSON)

Parse JSON without schema

```
import { JsonOutputParser } from "@langchain/core/output_parsers";

const parser = new JsonOutputParser();

const chain = prompt | model | parser;

const result = await chain.invoke({
  input: "Return JSON with category and amount for: Lunch ₹500"
});
```

```
console.log(result);
// { category: "Food", amount: 500 }  ← Parsed JSON
```

19.5 Comma-Separated List Parser

Parse comma-separated values

```
import { CommaSeparatedListOutputParser } from "@langchain/core/output_parsers";

const parser = new CommaSeparatedListOutputParser();

const chain = prompt | model | parser;

const result = await chain.invoke({
  input: "List 3 expense categories"
});

console.log(result);
// ["Food", "Transport", "Shopping"]  ← Array of strings
```

19.6 OutputFixingParser (Auto-Fix)

Automatically fix malformed output

```
import { OutputFixingParser } from "langchain/output_parsers";

const baseParser = StructuredOutputParser.fromZodSchema(schema);

const fixingParser = OutputFixingParser.fromLLM(
  llm,
  baseParser
);

// If LLM returns invalid JSON
const malformed = '{ category: "Food", amount: 500 }';  // Missing quotes

// Base parser would fail, but fixing parser:
const fixed = await fixingParser.parse(malformed);
// Sends malformed output to LLM, asks it to fix, then parses
```

☑ **You now understand Output Parsers!**

Chapter 20: Callbacks - Event Tracking

20.1 What Are Callbacks?

Callbacks = Event listeners for LangChain operations (logging, monitoring, streaming)

Track events like:

- 🚀 Chain/LLM starts
- 📝 LLM tokens generated
- ✅ Chain/LLM ends
- ❌ Errors
- 🔧 Tool calls

20.2 Basic Callback Handler

```
import { BaseCallbackHandler } from "@langchain/core/callbacks";

class LoggingHandler extends BaseCallbackHandler {
  name = "LoggingHandler";

  async handleLLMStart(llm, prompts) {
    console.log("[LLM Start]", prompts);
  }

  async handleLLMEnd(output) {
    console.log("[LLM End]", output.generations[0][0].text);
  }

  async handleChainStart(chain, inputs) {
    console.log("[Chain Start]", chain.name, inputs);
  }

  async handleChainEnd(outputs) {
    console.log("[Chain End]", outputs);
  }

  async handleToolStart(tool, input) {
    console.log("[Tool Start]", tool.name, input);
  }

  async handleToolEnd(output) {
    console.log("[Tool End]", output);
  }
}

// Use in chain
const chain = prompt | model;
await chain.invoke(
  { input: "Hello" },
  { callbacks: [new LoggingHandler()] }
);
```

20.3 Streaming Callbacks

Stream tokens as generated

```
class StreamingHandler extends BaseCallbackHandler {
  async handleLLMNewToken(token) {
    process.stdout.write(token);  // Print each token immediately
  }
}

const chain = prompt | model;
await chain.invoke(
  { input: "Tell a story" },
  { callbacks: [new StreamingHandler()] }
);
// Output: "Once" "upon" "a" "time" ... (word by word)
```

20.4 ai-langx/ Cost Tracking Callback

```
// File: src/utils/costTracking.js
import { BaseCallbackHandler } from "@langchain/core/callbacks";

class CostTrackingHandler extends BaseCallbackHandler {
  name = "CostTrackingHandler";

  constructor() {
    super();
    this.totalTokens = 0;
    this.cost = 0;
  }

  async handleLLMEnd(output) {
    const usage = output.llmOutput?.tokenUsage || {};
    const { totalTokens = 0 } = usage;

    this.totalTokens += totalTokens;

    // GPT-4o-mini pricing: $0.15 / 1M input tokens, $0.60 / 1M output tokens
    const inputCost = ((usage.promptTokens || 0) / 1000000) * 0.15;
    const outputCost = ((usage.completionTokens || 0) / 1000000) * 0.60;
    this.cost += inputCost + outputCost;
  }

  getMetrics() {
    return {
      totalTokens: this.totalTokens,
      totalCost: this.cost.toFixed(4)
    };
  }
}

// Usage
const costTracker = new CostTrackingHandler();
await agent.invoke({ input: "..." }, { callbacks: [costTracker] });
```

```
console.log("Cost:", costTracker.getMetrics());
// { totalTokens: 1523, totalCost: "0.0012" }
```

☑ **You now understand Callbacks!**

Chapter 21: Caching - Performance Optimization

21.1 Why Caching?

Problem: Repeated LLM calls are slow and expensive

```
// Same question asked 100 times
for (let i = 0; i < 100; i++) {
  await llm.invoke("What's 2+2?");  // 100 API calls, $$$$
}
```

Solution: Cache responses

```
// First call: API request
const result1 = await cachedLLM.invoke("What's 2+2?");  // API call

// Subsequent calls: Instant from cache
const result2 = await cachedLLM.invoke("What's 2+2?");  // Cache hit!
```

21.2 InMemoryCache

Simple in-memory caching

```
import { ChatOpenAI } from "@langchain/openai";

const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  cache: true  // ←  Enable cache
});

// First call: Slow (API request)
const start1 = Date.now();
await llm.invoke("What's the capital of France?");
console.log(`Time: ${Date.now() - start1}ms`);  // ~1000ms

// Second call: Fast (cache hit)
const start2 = Date.now();
await llm.invoke("What's the capital of France?");
console.log(`Time: ${Date.now() - start2}ms`);  // ~5ms ☑
```

21.3 Redis Cache (Persistent)

Cache across restarts

```
import { Redis } from "ioredis";
import { RedisCacheStore } from "@langchain/community/caches/ioredis";

const redis = new Redis(process.env.REDIS_URL);

const cache = new RedisCacheStore({ client: redis });

const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  cache
});

// Cached in Redis (persists across restarts)
```

21.4 Cache-Backed Embeddings

Cache expensive embeddings

```
import { CacheBackedEmbeddings } from "langchain/embeddings/cache_backed";
import { InMemoryCacheStore } from "@langchain/community/caches/in_memory";

const underlyingEmbeddings = new OpenAIEmbeddings();
const cacheStore = new InMemoryCacheStore();

const cachedEmbeddings = CacheBackedEmbeddings.fromBytesStore(
  underlyingEmbeddings,
  cacheStore,
  { namespace: "expense-embeddings" }
);

// First embed: Calls API
await cachedEmbeddings.embedQuery("lunch expense");  // ~200ms

// Second embed: From cache
await cachedEmbeddings.embedQuery("lunch expense");  // ~1ms ☑
```

☑ **You now understand Caching!**

Chapter 22: Advanced Retrievers

22.1 Self-Query Retriever

LLM generates filter from natural language

```
import { SelfQueryRetriever } from "langchain/retrievers/self_query";

const retriever = SelfQueryRetriever.fromLLM({
  llm,
  vectorStore,
  documentContents: "Expense documents",
  attributeInfo: [
    { name: "category", description: "Expense category (Food, Transport, etc.)",
type: "string" },
    { name: "amount", description: "Expense amount", type: "number" },
    { name: "date", description: "Expense date", type: "string" }
  ]
});

// User query (natural language)
const docs = await retriever.getRelevantDocuments(
  "Show me food expenses over ₹500 from last month"
);

// Behind the scenes:
// 1. LLM converts to filter: { category: "Food", amount: { $gt: 500 }, date: {
$gte: "2026-01-01" } }
// 2. Applies filter to vector search
// 3. Returns matching docs
```

22.2 Time-Weighted Retriever

Prioritize recent documents

```
import { TimeWeightedVectorStoreRetriever } from
"langchain/retrievers/time_weighted";

const retriever = new TimeWeightedVectorStoreRetriever({
  vectorStore,
  memoryStream: [],  // Stores access times
  searchKwargs: { k: 3 },
  decayRate: 0.01  // How quickly old docs lose relevance
});

// Recent docs ranked higher even if slightly less similar
```

22.3 Parent Document Retriever

Retrieve small chunks, return full parent documents

```
import { ParentDocumentRetriever } from "langchain/retrievers/parent_document";

// Small chunks for precise retrieval
```

```
const childSplitter = new RecursiveCharacterTextSplitter({
  chunkSize: 400
});

// Large chunks for context
const parentSplitter = new RecursiveCharacterTextSplitter({
  chunkSize: 2000
});

const retriever = new ParentDocumentRetriever({
  vectorstore,
  docstore,  // Stores parent documents
  childSplitter,
  parentSplitter
});

// Searches with small chunks (precise)
// Returns large parents (full context)
```

☑ **You now understand Advanced Retrievers!**

Chapter 23: Advanced Chain Patterns

23.1 Summarization Chain

Summarize long documents

```
import { loadSummarizationChain } from "langchain/chains";

const chain = loadSummarizationChain(llm, {
  type: "map_reduce",  // Or "stuff", "refine"
});

const docs = [/* long documents */];
const result = await chain.call({ input_documents: docs });

console.log(result.text);  // Summary
```

Types:

- **"stuff"**: All docs in one prompt (fast, but limited by context)
- **"map_reduce"**: Summarize each doc, then summarize summaries (scalable)
- **"refine"**: Iterative refinement (best quality)

23.2 ConstitutionalChain (Self-Critique)

LLM critiques and improves its own output

```
import { ConstitutionalChain, ConstitutionalPrinciple } from "langchain/chains";

const chain = ConstitutionalChain.fromLLM(llm, {
  critiqueLLM: llm,
  constitutionalPrinciples: [
    new ConstitutionalPrinciple({
      name: "Polite",
      critiqueRequest: "Does this response sound polite and professional?",
      revisionRequest: "Revise to be more polite and professional."
    })
  ]
});

// Initial response: "Give me your expenses."
// After critique: "Could you please share your expense information?"
```

23.3 API Chain (Call External APIs)

Chain that calls APIs

```
import { APIChain } from "langchain/chains";

const chain = new APIChain({
  llm,
  apiDocs: `
API: GET /expenses?user_id={id}
Returns: JSON array of expenses
`,
  headers: {
    Authorization: `Bearer ${API_KEY}`
  }
});

const result = await chain.call({
  question: "What are my recent expenses?"
});
// Chain calls API, parses response, answers question
```

23.4 Router Chain (Multi-Prompt)

Route to different prompts based on input

```
import { MultiPromptChain } from "langchain/chains";

const chain = MultiPromptChain.fromPrompts(llm, {
  expense_query: {
    template: "Answer expense question: {input}",
    description: "For questions about expenses"
```

```
    },
    approval_request: {
      template: "Process approval request: {input}",
      description: "For approval requests"
    }
  });

  // Automatically routes to correct prompt
```

☑ **You now understand Advanced Chains!**

Part 4 Complete Summary

Concepts Covered: 50+

LCEL (10+ concepts)

- ☑ What LCEL is and benefits
- ☑ Pipe operator and composition
- ☑ invoke(), batch(), stream()
- ☑ vs old LangChain v0.0 syntax

Runnables (10+ concepts)

- ☑ Runnable interface
- ☑ RunnablePassthrough, RunnableLambda
- ☑ RunnableBranch (conditional)
- ☑ RunnableParallel/RunnableMap
- ☑ Fallbacks and retry logic
- ☑ Custom runnables

Output Parsers (8+ concepts)

- ☑ Why parsers needed
- ☑ StringOutputParser
- ☑ StructuredOutputParser (Zod)
- ☑ JsonOutputParser
- ☑ CommaSeparatedListOutputParser
- ☑ OutputFixingParser

Callbacks (5+ concepts)

- ☑ What callbacks are
- ☑ BaseCallbackHandler
- ☑ Streaming callbacks
- ☑ Cost tracking
- ☑ Custom event handlers

Caching (5+ concepts)

- ☑ Why cache
- ☑ InMemoryCache
- ☑ RedisCache
- ☑ Cache-backed embeddings
- ☑ Cache invalidation

Advanced Retrievers (5+ concepts)

- ☑ Multi-Query retriever
- ☑ Self-Query retriever
- ☑ Time-Weighted retriever
- ☑ Parent Document retriever
- ☑ Contextual Compression retriever

Advanced Chains (7+ concepts)

- ☑ Summarization chains (map_reduce, stuff, refine)
- ☑ ConstitutionalChain
- ☑ API Chain
- ☑ Router chains
- ☑ SQL chains
- ☑ Custom chain patterns

Hands-On Challenge: Production-Ready Pipeline

Build expense processing with all advanced features:

1. **LCEL chain** with fallbacks
2. **Structured output** with Zod validation
3. **Caching** for repeated queries
4. **Callbacks** for cost tracking
5. **Error handling** with retries

Starter:

```
const costTracker = new CostTrackingHandler();

const chain = (prompt | primaryModel | parser)
  .withFallbacks({ fallbacks: [fallbackModel | parser] })
  .withRetry({ stopAfterAttempt: 3 });

const result = await chain.invoke(
  { input: "..." },
  { callbacks: [costTracker] }
);

console.log("Result:", result);
console.log("Cost:", costTracker.getMetrics());
```

Continue to Part 5: PART_5_LANGGRAPH.md (StateGraph Workflows - 40+ concepts)

Or Part 6: PART_6_LANGSMITH.md (Observability & Monitoring - 40+ concepts)